

EXPERIMENT – 4

Experiment Title:

Analysis of Pointers, Memory Management, and Registers using GDB

Aim:

To study and analyze Pointer behavior in C, Memory management (stack, heap, data segment) and CPU registers and execution flow using the GDB Debugger

Software Used:

C Programming Language, GCC Compiler (gcc), GDB Debugger and Visual Studio Code / Terminal

Theory:

Pointers in C

A pointer stores the memory address of another variable.

- & → Address-of operator
- * → Dereference operator

Example:

```
int a = 5;
```

```
int *p = &a;
```

Memory Management

A C program uses different memory segments:

- Stack - Local variables, function calls
- Heap - Dynamic memory (malloc, free)
- Data Segment - Global/static variables
- Code Segment - Program instructions

CPU Registers

Registers are high-speed memory inside CPU:

- EAX, EBX, ECX, EDX → General purpose
- ESP → Stack pointer
- EBP → Base pointer
- EIP → Instruction pointer

GDB Debugger

GDB allows:

- Step-by-step execution
- Breakpoints
- Register inspection
- Memory analysis

Program Code:

Part A:

```
#include <stdio.h>

int main() {
    int int_var = 5;
    int *int_ptr;

    int_ptr = &int_var; // Put the address of int_var into int_ptr

    printf("int_var = %d\nint_var = %p\n\n", int_var, &int_var);
    printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n\n", int_ptr, &int_ptr, *int_ptr);

    *int_ptr = 10; // Change the value via the pointer

    printf("int_var = %d\nint_var = %p\n\n", int_var, &int_var);
    printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n", int_ptr, &int_ptr, *int_ptr);

    return 0;
}
```

Part B:

```
#include <stdio.h>
#include <stdlib.h>

int global = 3; // Global Segment

int main() {
    char hello[] = "hello"; // Stack (Array)
    char* hello2 = "hello2"; // Value is in Text, Pointer is on Stack

    int* p = malloc(10); // Allocated on Heap

    printf("hello= %p\n", (void*)hello);
    printf("hello2= %p\n", (void*)hello2);
    printf("&hello2= %p\n", (void*)&hello2);

    printf("main= %p\n", (void*)main);
    printf("&global= %p\n", (void*)&global);
    printf("p= %p\n", (void*)p);

    free(p);
    return 0;
}
```

Procedure:

Part A:

- Write pointer program
- Compile:

gcc -g pointer.c -o pointer

- Run:

./pointer

- Observe:
 - Variable values
 - Memory addresses
 - Pointer dereferencing

Part B :

- Compile program:

gcc -g assignment.c -o assignment

- Start GDB:

gdb ./assignment

- Execute commands:
 - break main → Set breakpoint
 - run → Start execution
 - n → Step execution
 - info registers → View registers
 - print &var → Address of variable
 - print p → Pointer value
 - x → Examine memory

Output & Debugging:

Part A

```
● PS C:\Users\LAB 102 PC 11\Desktop\VS CODE> gcc -g pointer.c -o pointer
● PS C:\Users\LAB 102 PC 11\Desktop\VS CODE> ./pointer
int_var = 5
&int_var = 0061FF1C

int_ptr = 0061FF1C
&int_ptr = 0061FF18
*int_ptr = 5

int_var = 10

int_var = 10
int_var = 10
&int_var = 0061FF1C
&int_var = 0061FF1C
○

int_ptr = 0061FF1C
int_ptr = 0061FF1C
&int_ptr = 0061FF18
*int_ptr = 10
*int_ptr = 10
```

```
PS C:\Users\LAB 102 PC 11\Desktop\VS CODE> gdb ./pointer
PS C:\Users\LAB 102 PC 11\Desktop\VS CODE> gdb ./pointer
GNU gdb (GDB) 7.6.1
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
and "show warranty" for details.
This GDB was configured as "mingw32".
This GDB was configured as "mingw32".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from C:\Users\LAB 102 PC 11\Desktop\VS CODE\pointer.exe...done.
```

```
(gdb) n
7      int_ptr = &int_var; // Put the address of int_var into int_ptr
(gdb) n
(gdb) n
9      printf("int_var = %d\n&int_var = %p\n\n", int_var, &int_var);
(gdb) n
(gdb) n
9      printf("int_var = %d\n&int_var = %p\n\n", int_var, &int_var);
(gdb) n
(gdb) n
int_var = 5
&int_var = 0061FF1C
int_var = 5
&int_var = 0061FF1C

10     printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n\n", int_ptr, &int_ptr, *int_ptr);
10     printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n\n", int_ptr, &int_ptr, *int_ptr);
(gdb) n
(gdb) n
int_ptr = 0061FF1C
int_ptr = 0061FF1C
&int_ptr = 0061FF18
&int_ptr = 0061FF18
*int_ptr = 5
*int_ptr = 5

12     *int_ptr = 10; // Change the value via the pointer
```

```

(gdb) n
7      int_ptr = &int_var; // Put the address of int_var into int_ptr
(gdb) n
(gdb) n
9      printf("int_var = %d\n&int_var = %p\n\n", int_var, &int_var);
(gdb) n
(gdb) n
9      printf("int_var = %d\n&int_var = %p\n\n", int_var, &int_var);
(gdb) n
(gdb) n
int_var = 5
&int_var = 0061FF1C
int_var = 5
&int_var = 0061FF1C

10      printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n\n", int_ptr, &int_ptr, *int_ptr);

10      printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n\n", int_ptr, &int_ptr, *int_ptr);
(gdb) n
(gdb) n
int_ptr = 0061FF1C
&int_ptr = 0061FF1C
&int_ptr = 0061FF18
&int_ptr = 0061FF18
*int_ptr = 5
*int_ptr = 5

12      *int_ptr = 10; // Change the value via the pointer

```

```

(gdb) n
12      *int_ptr = 10; // Change the value via the pointer
(gdb) n
14      printf("int_var = %d\n&int_var = %p\n\n", int_var, &int_var);
(gdb) n
int_var = 10
&int_var = 0061FF1C
&int_var = 0061FF1C

15      printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n", int_ptr, &int_ptr, *int_ptr);
15      printf("int_ptr = %p\n&int_ptr = %p\n*int_ptr = %d\n", int_ptr, &int_ptr, *int_ptr);
(gdb) n
int_ptr = 0061FF1C
&int_ptr = 0061FF18
*int_ptr = 10
17      return 0;
(gdb) n
18      }

```

```

Starting program: C:\Users\LAB 102 PC 11\Desktop\VS CODE\./pointer.exe
[New Thread 10384.0x4614]
[New Thread 10384.0x4a90]

Breakpoint 1, main () at pointer.c:4
4      int int_var = 5;
(gdb) x/4xw &int_var
0x61ff1c:      0x002bf000      0x00400080      0x0061ff10      0x0061ff54
(gdb) continue
Continuing.
int_var = 5
&int_var = 0061FF1C

int_ptr = 0061FF1C
&int_ptr = 0061FF18
*int_ptr = 5

int_var = 10
&int_var = 0061FF1C

int_ptr = 0061FF1C
&int_ptr = 0061FF18
*int_ptr = 10
[Inferior 1 (process 10384) exited normally]

```

Part B:

```
• PS C:\Users\LAB 102 PC 11\Desktop\VS CODE> gcc -g assignment.c -o assignment
• PS C:\Users\LAB 102 PC 11\Desktop\VS CODE> ./assignment
hello= 0061FF16
hello2= 00405064
&hello2= 0061FF10
main= 00401460
&global= 00404004
p= 00711AB0
```

```
○ PS C:\Users\LAB 102 PC 11\Desktop\VS CODE> gdb ./assignment
GNU gdb (GDB) 7.6.1
Copyright (C) 2013 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law. Type "show copying"
and "show warranty" for details.
This GDB was configured as "mingw32".
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>...
Reading symbols from C:\Users\LAB 102 PC 11\Desktop\VS CODE\assignment.exe...done.
(gdb) break 20
Breakpoint 1 at 0x40150d: file assignment.c, line 20.
(gdb) run
Starting program: C:\Users\LAB 102 PC 11\Desktop\VS CODE\./assignment.exe
[New Thread 19088.0x49ac]
[New Thread 19088.0x27e4]
hello= 0061FF16
hello2= 00405064
&hello2= 0061FF10
main= 00401460
&global= 00404004
p= 008F1AD8

Breakpoint 1, main () at assignment.c:20
20      free(p);
```

```
(gdb) info registers rip
Invalid register `rip'
hello= 0061FF16
hello2= 00405064
&hello2= 0061FF10
main= 00401460
&global= 00404004
p= 008F1AD8

Breakpoint 1, main () at assignment.c:20
20      free(p);
```

```
(gdb) info registers
eax          0x12      18
ecx          0x23b1640b    598828043
edx          0x30000    196608
ebx          0x321000    3280896
esp          0x61ff00    0x61ff00
eax          0x12      18
ecx          0x23b1640b    598828043
edx          0x30000    196608
ebx          0x321000    3280896
esp          0x61ff00    0x61ff00
ebx          0x321000    3280896
esp          0x61ff00    0x61ff00
ebp          0x61ff28    0x61ff28
esi          0x4012e0    4199136
ebp          0x61ff28    0x61ff28
esi          0x4012e0    4199136
edi          0x4012e0    4199136
eip          0x40150d    0x40150d <main+173>
eflags      0x246      [ PF ZF IF ]
edi          0x4012e0    4199136
eip          0x40150d    0x40150d <main+173>
eflags      0x246      [ PF ZF IF ]
eip          0x40150d    0x40150d <main+173>
eflags      0x246      [ PF ZF IF ]
cs          0x23      35
cs          0x23      35
ss          0x2b      43
ss          0x2b      43
ds          0x2b      43
es          0x2b      43
fs          0x53      83
gs          0x2b      43
```

```
(gdb) print &global
$1 = (int *) 0x404004 <global>
(gdb) print p
$2 = (int *) 0x8f1ad8
(gdb) print &hello
$3 = (char (*)[6]) 0x61ff16
(gdb) █
```

Observations:

- Pointer stores address of variable
- Dereferencing accesses actual value
- Changing *ptr updates original variable
- Stack stores local variables
- Heap stores dynamically allocated memory
- Global variables stored in data segment
- Registers control execution flow

- EIP shows current instruction

Result:

- Successfully demonstrated pointer operations
- Observed memory allocation in stack, heap, and data segments
- Analyzed CPU registers using GDB
- Verified program execution step-by-step

Conclusion:

This experiment provided a comprehensive understanding of pointers, memory organization, and CPU registers. Using GDB debugger, the internal working of program execution was analyzed effectively, helping in understanding how memory and registers interact during execution.

Author: Aayush Anand